Arithmetic Expression Evaluator
Software Architecture Document

Version 1.1

Revision History

Date	Version	Description	Author
4/07/2026	1.0	Organized and adjusted the provided template structure for the project	RG
4/08/2026	1.0	Filled in Introduction, Scope, and Overview based on project requirements	Mahathi
4/08/2026	1.0	Added initial content for Goals, Constraints, and Logical View	Ayesha
4/10/2026	1.1	Developed Process View and Implementation View sections	RG
4/10/2026	1.1	Expanded and finalized module explanations	Ayesha
4/11/2026	1.1	Reviewed overall structure and consistency with architecture. Edited wording, clarity, and formatting. Finalized Quality Attributes and Risks/Tradeoffs	Mahathi

Table of Contents

1. Introduction 4
   - 1.1 Purpose 4
   - 1.2 Scope 4
   - 1.3 Definitions, Acronyms, and Abbreviations 4
   - 1.4 References 4
   - 1.5 Overview 4

2. Architectural Representation 5

3. Architectural Goals and Constraints 4

4. Use-Case View 6
   - 4.1 Use-Case Realizations 6

5. Logical View 6
   - 5.1 Overview 6
   - 5.2 Architecturally Significant Design Packages 6

6. Interface Description 7

7. Size and Performance 7

8. Quality 8

Software Architecture Document

1. Introduction

This document presents the overall architecture of the Arithmetic Expression Evaluator project. Its purpose is to clearly describe how the system is structured, how its main components interact, and the reasoning behind the chosen design. By outlining the architecture, this document helps ensure that all team members share a common understanding of the system and can work on different parts efficiently.

1.1.2 Purpose

This document describes the high-level architecture of our Arithmetic Expression Evaluator project. We wrote it to explain how the system is organized, how the major modules interact, and why this structure fits the project requirements.

1.1.3 Scope

Our project is a C++ application that accepts arithmetic expressions, parses them, and computes the result. The system supports `+`, `-`, `*`, `/`, `%`, and `**`, along with unary operators and parentheses. The architecture also supports the team's extended GUI features, such as parse tree visualization, token stream display, and step-by-step evaluation.

1.1.4 Definitions, Acronyms, and Abbreviations

TERMS	MEANING
AST	Abstract Syntax Tree
GUI	Graphical User Interface
CLI	Command-Line Interface
PEMDAS	Standard order of operations
Qt	Cross Platform application framework
CMake	Build system generator

References Documents:

- EECS 348 Term Project Description
- Software Requirements Specification
- Project Plan
- Programming Guidelines
- README in the project repository

1.1.5 Overview

The rest of this document explains the chosen architecture, the major modules, how the system flows at runtime, how the code is organized, how it will be deployed, and what risks and tradeoffs we considered.

2. Architectural Representation

The system uses a modular layered architecture that separates input, tokenization, parsing, evaluation, and output. This represents how the system is structured and how expressions are processed in stages.

The architecture is represented through multiple views: logical, process, implementation, deployment, data, and use-case. The logical view includes system components, the process view shows data flow, the implementation view shows code structure, the deployment view shows the runtime environment, the data view shows data elements, and the use-case view shows user interactions.

2.1.1 Chosen Architecture

We chose a **modular layered architecture** for this project. The system naturally breaks into stages: input, tokenization, parsing, evaluation, and output. Keeping each stage in its own layer makes the project easier to build, test, and maintain.

2.1.2 Why We Chose It

This architecture fits our project because it supports separation of concerns. The tokenizer does not need to know anything about the GUI, and the evaluator does not need to know how the expression was entered. That keeps the code cleaner and makes future changes easier.

2.1.3 Architectural Summary

The user interface sends an input expression to the core logic. The tokenizer converts the expression into tokens, the parser builds a structured representation, the evaluator computes the result, and the error handler reports any problems. Optional visualization modules can show internal details without changing the core evaluation logic.

3. Architectural Goals and Constraints

Our main goals are to:
- Evaluate arithmetic expressions correctly.
- Handle operator precedence and association properly.
- Keep the code modular and easy to understand.
- Make the program easy to test.
- Support a user-friendly interface.
- Leave room for future enhancements.

3.1.1 Constraints

We must implement the project in C++. The project also needs to follow the course requirements, which include clear documentation, unit tests, and proper error handling. Our current repository direction uses Qt, CMake, and GitHub, so the architecture must support those tools.

3.1.2 Quality Priorities

Correctness is our top priority because the evaluator must produce the right answer for valid expressions. After that, maintainability and testability matter most because we want a design that all team members can work with comfortably. Usability is also important because the interface should be simple for students to use.

4. Use-Case View

This section describes the key use cases of the system that represent its main functionality. These use cases illustrate how users interact with the Arithmetic Expression Evaluator and how the system responds to different inputs, including valid expressions, invalid expressions, and requests for additional details. The purpose of this section is to demonstrate how the system's architecture supports real user scenarios and ensures correct and efficient processing of arithmetic expressions.

4.1.1 Use-Case Realizations

The system processes arithmetic expressions through a structured pipeline consisting of tokenization, parsing, evaluation, and output.

4.1.1 Use Case: Evaluate a Valid Expression

- The user enters a valid arithmetic expression.
- The system receives input through the UI.
- The tokenizer converts the input into tokens.
- The parser validates the syntax and builds an expression tree (AST).
- The evaluator computes the result using the tree.
- The final result is displayed to the user.

4.1.2 Use Case: Reject an Invalid Expression

- The user enters an invalid expression.
- The tokenizer or parser detects an error.
- The system stops further processing.
- The error handler generates an appropriate message.
- The UI displays the error message to the user.

4.1.3 Use Case: View Expression Details

- The user evaluates an expression.
- The system processes the input normally.
- The user requests additional details.
- The system displays token streams, parse trees, or step-by-step evaluation.

5. Logical View

5.1.1 Overview

The system follows a modular layered design where each component is responsible for a specific task. This separation of concerns improves maintainability, readability, and ease of testing

5.1.2 Architecturally Significant Design Modules or Packages

- **Input Module**
  Handles input from the user interface.
- **Tokenizer Module**
  Converts the raw input string into tokens such as numbers, operators, and parentheses.
- **Parser Module**
  Analyzes the sequence of tokens and builds an Abstract Syntax Tree (AST).

- **Evaluator Module**
  Traverses the AST and computes the final result based on operator precedence.
- **Error Handler Module**
  Detects and reports errors such as invalid syntax or division by zero.
- **User Interface Module**
  Provides interaction between the user and the system.
- **Visualization Module**
  Displays internal structures such as parse trees and token streams.
- **Utility Module**
  Provides shared helper functions used across different components.

6. Interface Description

The system provides both a Graphical User Interface (GUI) and a Command-Line Interface (CLI).

6.1.1 Inputs

1. Arithmetic expressions entered by the user

6.1.2 Outputs

2. Computed result
3. Error messages (e.g., invalid syntax, division by zero)

6.1.3 GUI Features

4. Input field for entering expressions
5. Display area for results
6. Optional visualization components (parse tree, tokens, evaluation steps)

6.1.4 CLI Features

7. Text-based input and output
8. Immediate display of results or error

7. Size and Performance

The system is a moderately sized academic project designed to efficiently evaluate arithmetic expressions. Performance considerations include:

- Efficient tokenization and parsing of input
- Fast evaluation using tree traversal
- Minimal delay in displaying results

The system is optimized for typical usage and is not intended for large-scale or high-performance computing tasks.

8. Quality

The architecture supports key quality attributes essential for the system.

8.1.1 Correctness

The system ensures accurate evaluation by clearly separating tokenization, parsing, and evaluation stages.

8.1.2 Maintainability

The modular design allows for easy updates and modifications without affecting the entire system.

8.1.3 Testability

Each component can be tested independently, enabling effective unit testing.

8.1.4 Usability

The interface is simple and user-friendly, allowing easy interaction and understanding of results.

8.1.5 Extensibility

The system can be extended to support additional operators or features with minimal changes